MAISON

Technical Architecture Document

Your Personal AI Fashion House — From Inspiration to Outfit

Document Version 1.0

July 2026

Status: Draft — for engineering review

Builds on: Product Discovery & Strategy · PRD & Agile Backlog · AI Feature & Responsible AI · Metrics, Experimentation & GTM

Table of Contents

1. System Overview

Purpose. This document specifies the technical architecture required to build MAISON — an AI-driven fashion styling assistant that converts an inspiration image or a plain-language brief into 2–3 complete, budget-appropriate outfits sourced across multiple retailers. It translates the product requirements in the PRD and the behavioural contract in the AI Feature & Responsible AI document into concrete services, data stores, models, and interfaces that engineering can build against.

Core job to be done: "I found a look I love — help me buy the whole thing, in my budget, wherever it's sold." The architecture exists to serve that sentence in under 15 seconds, with an interim acknowledgment inside 2 seconds, and without ever presenting an item that is not actually purchasable.

1.1 Design Tenets

- **Retailer-agnostic by construction.** The recommendation engine reasons over a unified, multi-retailer catalog. No single retailer's inventory is privileged in ranking logic — this is the direct technical counterpart to MAISON's core positioning claim.
- **Explainable over opaque.** Final ranking is rule-based and inspectable (weighted scoring), not an LLM black box. LLMs are used for understanding (vision/NLU) and explanation generation, never as the sole arbiter of which outfit wins.
- **Hybrid AI, not all-AI.** Vision + LLM handle unstructured input; deterministic filters and scoring handle business logic. This keeps cost, latency, and debuggability under control at MVP and at scale.
- **MVP-cost discipline, scale-ready seams.** Every layer has a cheap/managed MVP option and a documented upgrade path — no component requires a rewrite to scale, only a swap of its implementation (see Section 10 and 11).
- **Privacy and safety by design.** No facial recognition, no body-shape inference, feature-extraction-only image handling, and a hard validation gate so nothing unpurchasable is ever displayed as available (Section 9).
- **Graceful degradation.** If the AI/catalog pipeline is slow or down, the system falls back to a small set of pre-curated, occasion-based outfit sets rather than failing outright (NFR4 in the PRD).

1.2 The Five Conceptual Layers

At the highest level of abstraction, MAISON is five layers wrapped around a feedback loop:

1. **Understanding** — turns an image and/or free text into a structured style/intent profile (Vision + NLU).
2. **Retrieval** — turns that profile into a candidate set of real, in-stock, in-budget products (vector search over the unified catalog).
3. **Recommendation** — turns candidate products into 2–3 complete, diverse, ranked outfits (rule-based scoring + diversity + personalization).
4. **Experience** — turns ranked outfits into a trustworthy UI: cards, prices, retailers, "why this outfit," buy links.
5. **Learning** — turns every rating, click, save, and skip into a signal that improves the next recommendation.

Sections 3–8 unpack each of these into concrete services, models, and data contracts. Section 2 shows how they compose into the overall system.

2. Architecture Diagram

The diagram below shows the full system at MVP-to-early-scale maturity: client apps, the API gateway, five core application services, the AI/ML layer, the data layer, and external integrations.

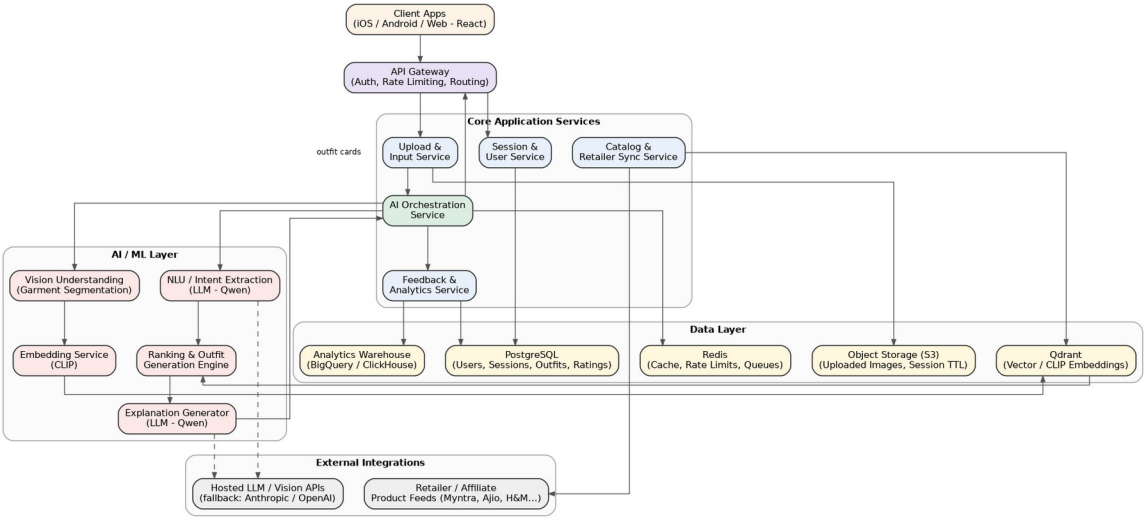

Figure 1 — MAISON high-level system architecture

2.1 Component Responsibility Map

Block	Responsibility	Primary Tech (MVP)
Client Apps	iOS / Android / responsive web capture UI; renders outfit cards, ratings, buy links	React Native or Flutter (mobile) + React web
API Gateway	AuthN/Z, rate limiting, request routing, versioning	Managed API Gateway (Kong / AWS API GW) or Nginx
Session & User Service	Auth, session/budget/size persistence, style profile	Node.js/Python service + PostgreSQL
Upload & Input Service	Accepts image/text, validates format & size, writes to object storage	Node.js/Python + S3-compatible storage
AI Orchestration Service	Sequences vision → NLU → retrieval → ranking → explanation; owns the 15s SLA and 2s ack	Python (FastAPI) orchestrator
Catalog & Retailer Sync Service	Ingests retailer/affiliate feeds, normalizes schema, refreshes stock/price	Scheduled workers + queue
Feedback & Analytics Service	Captures ratings, clicks, swaps; emits events to warehouse	Event pipeline (Kafka/SQS → warehouse)
AI / ML Layer	Vision segmentation, embeddings, NLU/LLM, ranking, explanation	See Section 4
Data Layer	Transactional, vector, cache, object, and analytical storage	See Section 6
External Integrations	Retailer/affiliate feeds; hosted LLM/vision APIs (fallback)	Retailer APIs, Anthropic/OpenAI APIs

3. Layer-by-Layer Design

3.1 Client Layer

Mobile-first (per persona research, both Riya and Neha are mobile-first, high tech-savviness users). A single capture screen exposes both image upload and a text field, per the A/B test design in the Metrics document (image-first vs. text-first framing is a UI ordering decision, not a separate client).

- Renders an interim state ("Maison is curating your look…") the moment the 2-second acknowledgment returns.
- Streams or polls for the final 2–3 outfit cards; swipeable on mobile, side-by-side on web/tablet.
- WCAG 2.1 AA: screen-reader descriptions for outfit images, high-contrast styling, keyboard-navigable comparison view.

3.2 API Gateway / BFF

A thin Backend-for-Frontend sits behind the gateway to shape payloads for mobile vs. web without leaking internal service topology to clients.

- AuthN via short-lived JWT + refresh token; session state cached in Redis for fast reads.
- Idempotency keys on the outfit-generation endpoint prevent duplicate expensive AI calls on client retries.
- Per-user and per-IP rate limiting to protect the AI layer from abuse and runaway cost.

3.3 Understanding Layer (Vision + NLU)

Consumes the raw image and/or text plus session context (occasion, budget, size, style history) and produces a structured style/intent profile — the JSON contract every downstream layer depends on.

- Vision path: garment/accessory detection and segmentation on the uploaded image, mapped to categories, colors, silhouette, and pattern.
- Text path: occasion, style register, and budget extraction from unstructured, sometimes ungrammatical or Hinglish, prompts.
- Output is a single normalized schema regardless of input modality, so Retrieval never needs to know whether the input was an image, text, or both.

Example structured output

```
{
  "occasion": "client_meeting",
  "style_register": "business_casual",
  "budget_inr": 5000,
  "garments_detected": [
    {"category": "top", "color": "beige", "silhouette": "relaxed"},
    {"category": "bottom", "color": "black", "silhouette": "tailored"}
  ],
  "size_profile": {"top": "M", "bottom": "30"},
  "confidence": 0.87
}
```

3.4 Retrieval Layer

Converts the style/intent profile into a candidate set of real products spanning every partnered retailer.

- Generates a CLIP embedding from the structured profile (image embedding when an image exists; text-to-CLIP embedding otherwise).
- Vector search (Qdrant) returns visually/stylistically similar products per garment category.
- A deterministic filter engine narrows on budget, size, availability, occasion tag, and retailer — yielding roughly 150–250 candidate products across 4–6 categories, matching the worked example in the source architecture notes (193 candidate products across 5 categories).

3.5 Recommendation Layer

Turns candidate products into complete outfits — this is the layer detailed fully in Section 8. In summary: generate 50–100 candidate outfit combinations, score them on a transparent weighted rubric, diversify the top set so three outfits feel meaningfully different, apply a small personalization boost, then hand the winners to the Explanation Generator.

3.6 Experience & Explainability Layer

Converts ranked, scored outfit data into the product surface the user actually sees: outfit cards with hero image, per-item retailer and price, total price, a plain-language "why this outfit" rationale, a confidence indicator, and the four action buttons (Buy Outfit, Save Outfit, Show Alternatives, Regenerate) shown in the user-flow diagram.

- The Alternative Generator re-queries Retrieval for a same-price-tier, in-stock substitute the moment an item goes out of stock, rather than breaking the whole outfit.
- Purchase flow stays retailer-agnostic: Buy Outfit → affiliate-tracked redirect → retailer checkout. MAISON never becomes a checkout platform at MVP.

3.7 Learning & Personalization Layer

Every rating, save, click, regenerate, and skip is logged as a signal (see Section 8.5) and folded into: (a) a lightweight per-user preference profile that boosts future scoring, and (b) an aggregate analytics stream that lets Product identify systematically weak occasion categories or catalog gaps.

3.8 Data & Infrastructure Layer

Underpins every layer above: PostgreSQL for transactional data, Qdrant for vector search, Redis for cache/session/queueing, S3-compatible object storage for uploaded images (short TTL), and an analytics warehouse for the event stream. Full schema in Section 6.

4. AI Model Selection

MAISON needs four distinct AI capabilities, not one model doing everything: (1) understanding an image, (2) understanding free text, (3) finding visually/stylistically similar products, and (4) explaining a recommendation in plain language. This section justifies the specific model choices, gives free/open and paid/hosted alternatives for each, and lays out the fine-tuning roadmap.

4.1 Why CLIP for Product & Style Embeddings

CLIP (Contrastive Language–Image Pre-training) learns a single joint embedding space for images and text. That single property is what makes it the right fit for MAISON, for three reasons:

- **One embedding space, two entry points.** A user can start from an image ("this Pinterest screenshot") or from text ("business casual under ₹5,000") and both land in the same vector space, so Retrieval doesn't need a separate index or a separate matching algorithm per input modality.
- **Off-the-shelf and cheap to run.** CLIP is small (86M–428M parameters depending on variant), runs comfortably on a single GPU or even CPU for MVP volumes, and needs no training data to get started — a pretrained checkpoint is usable on day one.
- **Compact vectors suit the vector DB.** 512-dimension embeddings keep Qdrant's index small and queries fast even as the catalog grows into the hundreds of thousands of SKUs.

Model choices

Stage	Model (Hugging Face)	License / Cost	Notes
MVP baseline	openai/clip-vit-base-patch32	Open weights, free to self-host	Fast, ~150M params, good enough to validate retrieval quality before investing further
Higher accuracy	openai/clip-vit-large-patch14	Open weights, free to self-host	Better fine-grained style discrimination; ~3–4× the compute of base
Fashion-specialized (recommended)	patrickjohncyh/fashion-clip	Open weights, free to self-host	CLIP fine-tuned on ~800K fashion image–caption pairs; materially better at garment attributes (fit, pattern, occasion) than generic CLIP out of the box
Managed / paid alternative	AWS Titan Multimodal Embeddings or Google Vertex Multimodal Embeddings	Pay-per-call	Removes GPU-hosting burden; reasonable MVP shortcut if the team has no ML-infra bandwidth yet

Recommendation: start with `patrickjohncyh/fashion-clip` self-hosted behind a small inference service. It is already fashion-domain-tuned, avoiding an early fine-tuning project, while remaining free and open.

4.2 Why Qwen for NLU, Outfit Reasoning, and Explanation

Three jobs need a language model: extracting occasion/style/budget from free text, reasoning over few-shot examples to assemble complementary items into candidate outfits, and turning structured scoring facts into a natural "why this outfit" explanation. Qwen (Alibaba's Qwen2.5 family) is the recommended base model family for the following reasons:

- **Open weights, permissive licensing.** Most Qwen2.5 sizes ship under an Apache 2.0-style license, so MAISON can self-host, fine-tune, and redistribute derivatives without per-token licensing risk — important once volume makes hosted-API cost a concern (Section 11).
- **A genuine range of sizes (SLM → LLM).** Qwen2.5 spans roughly 0.5B to 72B parameters in the same family, so the team can pick the smallest model that hits the required JSON-structure and reasoning accuracy, rather than defaulting to the largest available model.

- **Strong structured-output and instruction-following.** Outfit generation and NLU extraction both need reliable JSON output against a fixed schema — a place where Qwen2.5-Instruct models perform reliably with light prompt engineering.
- **Multilingual / Hinglish coverage.** MAISON's target users write casual, sometimes code-mixed Hindi-English prompts ("shaadi ke liar ₹6000 mein kuch"); Qwen's multilingual pretraining data gives it materially better coverage of this pattern than most Western-centric open models of comparable size.
- **A vision-language variant in the same family.** Qwen2.5-VL can directly describe garments from an image without a separate segmentation model — a useful MVP shortcut, traded off against precision in Section 4.4.

Model choices

Task	Model (Hugging Face)	Deployment	Why this size
NLU extraction (text → structured JSON)	Qwen2.5-7B-Instruct	Self-hosted (vLLM/TGI) or hosted inference	Small enough to be cheap and fast at MVP volume; strong enough for schema-constrained extraction
Outfit reasoning / few-shot assembly	Qwen2.5-14B-Instruct or Qwen2.5-32B-Instruct	Self-hosted, batched	More reasoning headroom for combining garments coherently; used behind the deterministic scorer, never as the final ranker (Section 4.5)
Explanation generation	Qwen2.5-7B-Instruct	Self-hosted	Explanation is a templated, low-risk generation task; the smallest capable model keeps unit cost near-zero
Image + text understanding (MVP shortcut)	Qwen2.5-VL-7B-Instruct	Self-hosted or hosted	Can replace a dedicated vision model at MVP if segmentation precision is not yet a bottleneck
Quality ceiling / cold-start (pre self-hosting)	Claude Haiku or GPT-4o-mini (hosted API)	Pay-per-token	Used during Wizard-of-Oz → early MVP to validate prompts fast without standing up GPU infra; migrate to self-hosted Qwen once volume crosses the cost break-even in Section 11

Recommendation: prototype and validate prompts against a hosted frontier API (Claude or GPT-4o-mini) during Wizard-of-Oz and early MVP for speed of iteration, then migrate the NLU-extraction and explanation-generation tasks to self-hosted `Qwen2.5-7B-Instruct` once monthly session volume makes self-hosting cheaper than per-token billing (worked example in Section 11).

4.3 Fine-Tuning Roadmap

Consistent with the product philosophy already stated in the discovery and AI documents ("MVP = simple, PM-chosen weights; V2 = feedback-driven; V3 = learned"), fine-tuning is staged rather than front-loaded:

Stage	What's fine-tuned	Method	Data source	Est. cost
MVP	Nothing — zero-/few-shot prompting only	Prompt engineering + structured-output validation	Hand-written few-shot examples	$0 (engineering time only)

Stage	What's fine-tuned	Method	Data source	Est. cost
V2	Qwen2.5-7B-Instruct for JSON-schema adherence + Indian fashion vocabulary	LoRA / QLoRA (parameter-efficient fine-tuning) via Hugging Face PEFT + TRL	Rated Wizard-of-Oz sessions + early production (prompt, outfit-JSON) pairs	~$50–300 on a rented A100/L4 for a few epochs
V3	fashion-clip contrastive fine-tune on MAISON's own catalog	Contrastive fine-tuning on (image, retailer-title) pairs, especially Indian ethnic-wear categories under-represented in Western fashion datasets	MAISON's growing multi-retailer catalog (needs ~50K+ SKUs to be worth it)	Low hundreds of $ in compute; main cost is data curation
V3	Ranking replaces hand-set weights with Learning-to-Rank	Gradient-boosted trees (LightGBM/XGBoost) trained on purchase/save/click/rating/skip signals	Production feedback loop (Section 8.5)	Low — CPU-trainable, no GPU required

LoRA/QLoRA is specifically recommended over full fine-tuning because it updates a small adapter (typically <1% of model parameters) rather than the full weight matrix — this preserves the base model's general reasoning while specializing it, and keeps GPU-hours (and therefore cost) low enough to iterate weekly rather than quarterly.

4.4 Vision: Garment Detection / Segmentation

Option	Type	Cost	Trade-off
yolos-fashionpedia (e.g., valentinafeve/yolos-fashionpedia)	Open-weight object detector fine-tuned on the Fashionpedia dataset	Free, self-hosted	Purpose-built for garment/category detection; needs its own small inference service
Segment Anything Model (SAM / SAM2, Meta)	Open-weight generic segmentation	Free, self-hosted	Excellent boundaries, not garment-aware out of the box — usually paired with a classifier
Qwen2.5-VL-7B-Instruct	Open-weight vision-language model	Free, self-hosted (GPU)	Fastest to stand up (one model instead of two); less precise on fine-grained attributes than a dedicated detector
Hosted vision API (Claude / GPT-4o / Google Vision)	Managed, pay-per-call	Pay-per-token or per-image	Zero infra, highest ease of iteration; recurring per-request cost that grows linearly with volume

Recommendation: MVP uses a hosted vision API or `Qwen2.5-VL-7B-Instruct` for speed; migrate to a dedicated fine-tuned detector (`yolos-fashionpedia` or a custom head on Fashion-CLIP features) once garment-misidentification (Failure Case 1 in the AI/Responsible-AI document) shows up often enough in ratings/alternative-requests to justify the investment.

4.5 Why Not Use an LLM for Final Ranking

Design decision

Asking a general-purpose LLM "what's the best outfit?" is non-deterministic, hard to debug, inconsistent across near-identical inputs, comparatively expensive at scale, and difficult to A/B test. MAISON instead uses a hybrid pipeline: AI extracts structured features → a rule-based scorer (transparent, PM-owned weights) ranks outfits → an LLM only narrates the winning outfits. The ranking decision itself stays fully explainable and reproducible; only the language wrapped around it is generative.

4.6 Summary Decision Table

Capability	MVP choice	Scale choice	Free / Paid
Image ↔ text embeddings	patrickjohncyh/fashion-clip	Same, or contrastively fine-tuned on MAISON catalog	Free (self-hosted)
Garment detection	Hosted vision API or Qwen2.5-VL-7B	yolos-fashionpedia / custom detector	Paid → Free
Text NLU extraction	Hosted API (Claude Haiku / GPT-4o-mini)	Self-hosted Qwen2.5-7B-Instruct (LoRA fine-tuned)	Paid → Free
Outfit assembly reasoning	Qwen2.5-14B/32B-Instruct (hosted or self-hosted)	Same, fine-tuned	Paid or Free
Explanation generation	Qwen2.5-7B-Instruct	Same	Free (self-hosted)
Final outfit ranking	Rule-based weighted scoring (no model)	Learning-to-Rank (LightGBM/XGBoost)	Free
Vector search	Qdrant (open source)	Qdrant Cloud or self-hosted cluster	Free → Paid (managed)

5. Data Flow Diagram

The diagram traces one outfit-generation request end to end, matching the six-stage AI workflow defined in the AI Feature document (Ingestion → Context Assembly → AI Matching → Budget & Safety Filter → Structure & Retailer Guard → Outfit Card Display), expanded here to the underlying technical steps.

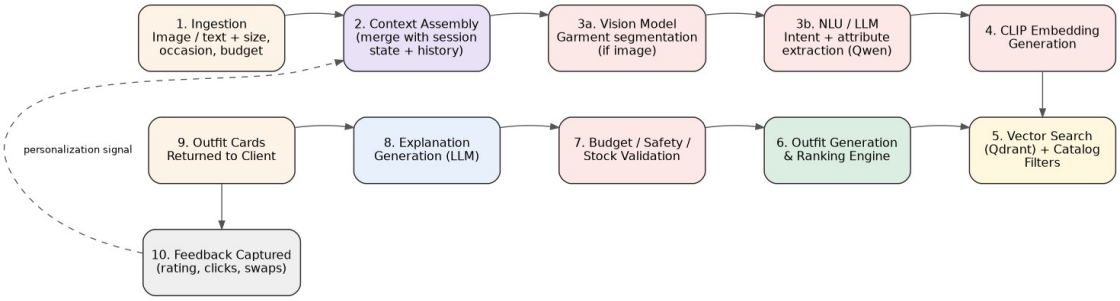

Figure 2 — End-to-end request data flow, with the feedback loop closing back into Context Assembly

5.1 Step-by-Step Narrative

1. **Ingestion.** Client submits an image, text, or both, plus optional size profile, to the Upload & Input Service. Images are validated (JPEG/PNG/WebP, ≤10MB) and written to short-TTL object storage.
2. **Context Assembly.** The AI Orchestration Service merges the raw input with session state — occasion, budget, size, and any style-history signal — into a single request payload.
3. **Vision / NLU.** If an image is present, it is segmented into garments; text (if present) is parsed for occasion, style register, and budget by the LLM. Both paths converge on the same structured schema.
4. **CLIP embedding.** The structured profile is embedded into the shared image/text vector space.
5. **Vector search + catalog filters.** Qdrant returns visually similar candidates per category; the filter engine narrows by budget, size, availability, occasion, and retailer.
6. **Outfit generation & ranking.** Candidate products become 50–100 candidate outfits, scored, diversified, and personalized (full detail in Section 8).
7. **Budget / safety / stock validation.** Every candidate outfit is re-checked against the hard budget ceiling, catalog freshness (≤24h), and content/IP safety before it is eligible to render — nothing unpurchasable is ever shown as available.
8. **Explanation generation.** The LLM converts each winning outfit's structured scoring facts into a short, plain-language “why this outfit” rationale.
9. **Outfit cards returned to client.** 2–3 outfit cards render with price, retailers, confidence, and action buttons, within the 15-second SLA (2-second acknowledgment shown earlier).
10. **Feedback captured.** Ratings, buy-link clicks, “Show Alternatives” taps, and out-of-stock swaps feed back into Context Assembly for the user's next interaction — the user is never made to start over.

6. Database Design

MAISON uses a polyglot persistence model: one system of record for transactional data, one purpose-built store for vector similarity, one cache/queue layer, one object store for images, and one append-only analytical store. This mirrors the fact that “find similar products,” “record a rating,” and “count funnel drop-off” are different access patterns that don't belong in the same engine.

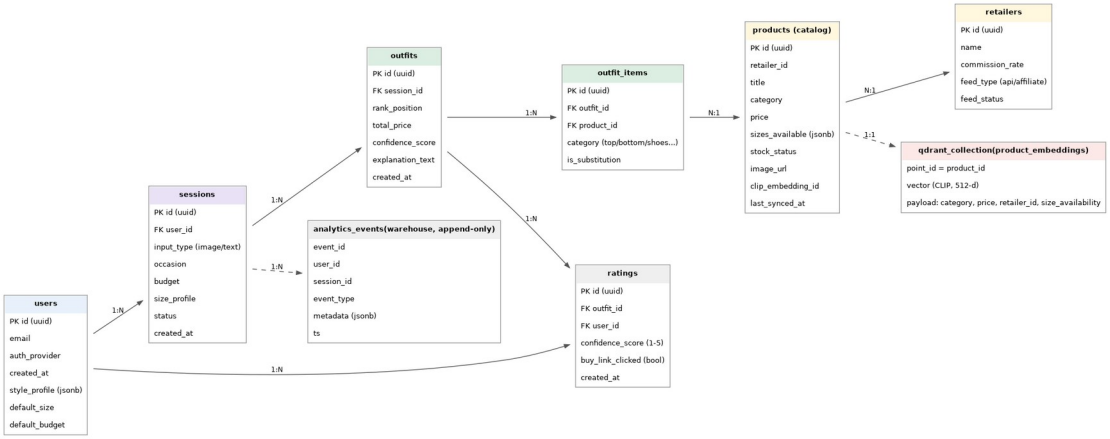

Figure 3 — Core relational schema, vector collection, and analytics event store

6.1 Core Tables (PostgreSQL)

Table	Key fields	Purpose
users	id, email, auth_provider, style_profile (jsonb), default_size, default_budget	Identity and the learned preference profile described in the AI document
sessions	id, user_id, input_type, occasion, budget, size_profile, status, created_at	One row per outfit-generation attempt
outfits	id, session_id, rank_position, total_price, confidence_score, explanation_text	One row per generated outfit (2–3 per session)
outfit_items	id, outfit_id, product_id, category, is_substitution	Line items within an outfit; is_substitution flags an auto-swapped item
products (catalog)	id, retailer_id, title, category, price, sizes_available (jsonb), stock_status, clip_embedding_id, last_synced_at	Normalized, multi-retailer product catalog
retailers	id, name, commission_rate, feed_type, feed_status	Partner metadata; commission_rate is normalized across retailers (Section 9.4 / GTM Section 7)
ratings	id, outfit_id, user_id, confidence_score (1–5), buy_link_clicked, created_at	Feeds the Decision Confidence Score north star metric

6.2 Vector Store (Qdrant)

A single collection, `product_embeddings`, stores one point per catalog product:

- point_id = product_id (foreign key back to PostgreSQL, never duplicated data).
- vector = 512-dimension CLIP (or Fashion-CLIP) embedding.

- payload = category, price, retailer_id, size_availability — enabling Qdrant's native payload filtering to combine similarity search with hard budget/size/retailer constraints in a single query.

6.3 Cache & Queue (Redis)

- Session state cache for fast reads during an active generation request.
- Rate-limit counters per user/IP.
- Lightweight job queue for catalog-sync workers and out-of-stock re-matching (can be upgraded to Kafka/SQS at scale, Section 10).

6.4 Object Storage & Analytics

- **Object storage (S3-compatible):** uploaded images with a short TTL, deleted at session end unless the user explicitly saves the outfit (post-MVP wardrobe feature) or requests deletion.
- **Analytics warehouse (append-only):** every event in the AARRR/event table from the Metrics document — Image Uploaded, Outfit Generated, Save Outfit, Buy Click, Rating Submitted, Alternative Requested — lands here for dashboards and A/B analysis, decoupled from the transactional path.

6.5 Free vs. Paid Options

Store	Free / self-hosted	Paid / managed	MVP recommendation
Relational (Postgres)	Self-hosted Postgres (Docker/VM)	Supabase, Neon, AWS RDS	Supabase or Neon — fast to stand up, generous free tier
Vector DB	Self-hosted Qdrant (open source)	Qdrant Cloud	Self-hosted for MVP volume; Qdrant Cloud once ops overhead matters
Cache / queue	Self-hosted Redis	Upstash, AWS ElastiCache	Upstash (serverless pricing fits spiky MVP traffic)
Object storage	MinIO (self-hosted)	AWS S3, Cloudflare R2	Cloudflare R2 — no egress fees, S3-compatible API
Analytics warehouse	Self-hosted ClickHouse	BigQuery, Snowflake	BigQuery pay-per-query — no infra to run at MVP scale

7. API Architecture

7.1 Style

REST over HTTPS with URI versioning (`/v1/...`), fronted by a thin BFF per client type. REST is chosen over GraphQL for MVP because the client's data needs are already well-defined and shallow (a session, its outfits, a rating) — GraphQL's flexibility is not yet worth its added gateway/query-complexity cost. Revisit GraphQL if the roadmap's wardrobe/calendar features (Section 3.7 of the Discovery doc) create many divergent client query shapes.

7.2 Core Endpoints

Endpoint	Method	Purpose
/v1/sessions	POST	Create a session: image and/or text, occasion, budget, size profile
/v1/sessions/{id}	GET	Poll session status (pending / curating / ready / failed-fallback)
/v1/sessions/{id}/outfits	GET	Fetch the 2–3 generated outfit cards once ready
/v1/outfits/{id}/alternatives	POST	Request a re-generation / “Show Alternatives” pass
/v1/outfits/{id}/ratings	POST	Submit a 1–5 confidence rating
/v1/outfits/{id}/buy-click	POST	Log a buy-link click for attribution, before redirect
/v1/catalog/sync-status	GET (internal)	Ops visibility into retailer feed freshness
/v1/webhooks/retailer-feed	POST (internal)	Retailer-side push notifications for stock/price changes, where supported

7.3 Handling the 2-Second Ack / 15-Second SLA

A synchronous request-response alone cannot satisfy both the 2-second acknowledgment and the up-to-15-second generation time gracefully. MAISON uses a two-phase pattern:

1. POST `/v1/sessions` returns within 2 seconds with a session id and status `"curating"` — this is what powers the “Maison is curating your look…” interim state.
2. The client either polls GET `/v1/sessions/{id}` on a short interval or subscribes via Server-Sent Events (SSE) for a push update the moment status flips to `"ready"` or `"fallback"` — SSE is preferred over WebSockets here since the flow is one-directional server → client.
3. If generation exceeds the SLA, the session status resolves to `"fallback"` and the pre-curated occasion-based outfit set is returned instead (NFR4), never a hard failure.

7.4 Cross-Cutting Concerns

- **Auth:** JWT bearer tokens; refresh via secure httpOnly cookie or mobile secure storage. Managed auth (Auth0, Clerk, Supabase Auth) is a reasonable paid shortcut at MVP to avoid building session/security primitives from scratch.
- **Idempotency:** Idempotency-Key header required on POST /v1/sessions to prevent duplicate, costly AI runs from client retries or double-taps.
- **Rate limiting:** per-user and per-IP limits at the gateway, tuned to the free-tier cap of 3 sessions/month (Metrics/GTM document, Section 7).
- **Versioning & deprecation:** breaking changes ship as /v2; /v1 is supported for a defined sunset window.

8. Recommendation Algorithm

This is the layer that turns “193 candidate products across 5 categories” into “3 outfits worth showing a person.” The guiding metaphor from the source architecture notes: MAISON doesn't pick the best goalkeeper, the best defender, and the best midfielder independently — it picks the best team.

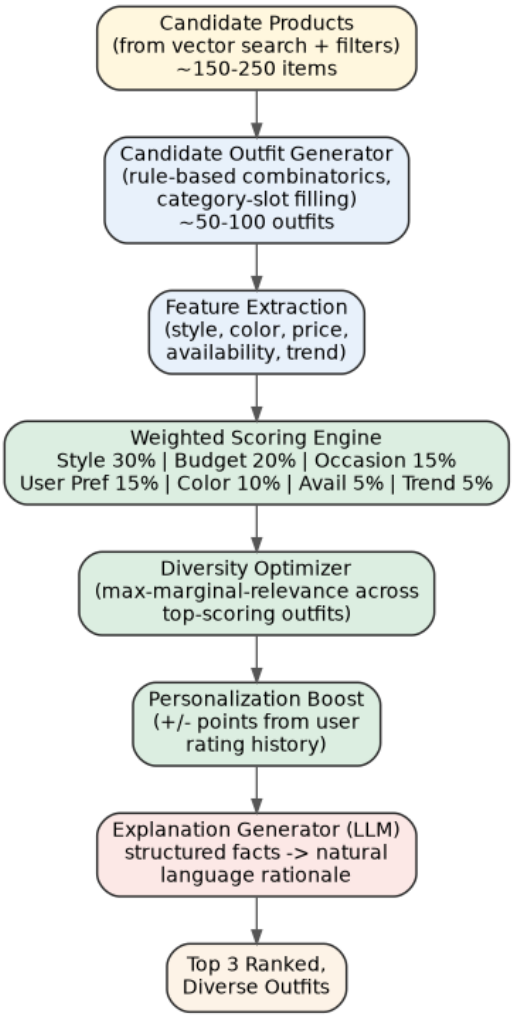

Figure 4 — Recommendation & ranking pipeline

8.1 Step 1 — Candidate Outfit Generation

Rather than generating a single “best” outfit, the engine assembles 50–100 candidate combinations by filling category slots (top, bottom, shoes, bag, accessory) from the retrieved candidate set, respecting hard constraints (budget ceiling, size availability) at construction time so no candidate is wasted on an outfit that can never be shown.

8.2 Step 2 — Weighted Scoring

Every candidate outfit receives an Overall Score as a weighted sum of independently PM-defined signals. This is a deliberate product decision, not a default: style is weighted above trend so a trendier-but-weaker-fit outfit does not beat a better-matched one.

Signal	Weight	What it captures
Style Match	30%	Alignment between detected/stated style and each item's attributes
Budget Match	20%	How efficiently the outfit uses the stated budget without exceeding it
Occasion Match	15%	Fit to the stated or inferred occasion
User Preferences	15%	Alignment with the learned preference profile (Section 8.4)
Color Harmony	10%	Palette coherence across items
Availability	5%	In-stock, in-size confidence
Trend Score	5%	Currency of the look — intentionally the lowest weight

8.3 Step 3 — Diversity Engine

The top-scoring outfits are frequently near-duplicates (same shirt, three shades of jeans). A diversity pass (maximal-marginal-relevance style re-ranking across the top-N candidates) ensures the final 2–3 outfits represent meaningfully different looks — e.g., Minimal / Elevated Casual / Smart Chic — while all still satisfying the original brief. This is the single highest-leverage step for perceived intelligence, since technically-different-but-practically-identical outfits read as a weak product.

8.4 Step 4 — Personalization Boost

A user's learned preference profile (neutral colors, a preferred brand, a disliked silhouette) contributes a small additive bonus — the source design intentionally caps this around +8 points on a 100-point scale so personalization nudges the ranking without letting it dominate style, budget, or occasion fit.

8.5 Step 5 — Confidence Score Definition

What “92% confidence” actually means

Confidence is not model certainty. It is defined as the weighted match between the user's expressed intent and the recommended outfit — the same weighted signals from Section 8.2, normalized to a percentage. “92%” means “we estimate this outfit satisfies 92% of your expressed preferences,” which is both more honest and easier to explain to a user than an opaque model-confidence number.

8.6 Step 6 — Explanation Generation

The LLM (Section 4.2) receives the structured scoring breakdown for the winning outfit — not a free-form prompt — and converts it into 3–5 short bullet points such as “Fits your ₹5,000 budget,” “Matches the oversized silhouette,” “Available in your size.” This keeps explanations grounded in the same facts that drove ranking, rather than inventing a rationale.

8.7 Feedback Signals

Signal type	Examples	Effect
Positive	Saved outfit, purchased outfit, clicked retailer, rated 5, viewed details	Reinforces the associated style/brand/color attributes in the

Signal type	Examples	Effect
		preference profile
Negative	Regenerated recommendations, rated 1, closed page, ignored outfit	Down-weights the associated attributes

8.8 MVP vs. V2 vs. V3

- **MVP:** simple weighted scoring with PM- and fashion-expert-chosen weights (Section 8.2 table) — easy to reason about and to adjust after each round of desirability testing.
- **V2:** adjust weights using aggregate feedback — e.g., if users consistently reject loafers, reduce that category's implicit score contribution.
- **V3:** Learning-to-Rank (gradient-boosted trees) trained on purchases, saves, clicks, ratings, and skips replaces hand-set weights — while explanation generation stays templated and auditable, preserving the "explainable ranking, generative narration" split from Section 4.5.

8.9 Why This Is Defensible

Any team can call a general-purpose LLM or stand up vector search. The combination of understanding + retrieval + an explainable, tunable ranking policy + personalization + a continuous learning loop is what is difficult to copy quickly — and it is also the reason the ranking logic deliberately does not live inside an LLM prompt that a competitor could simply replicate by copying the prompt.

9. Security & Privacy

9.1 Data Classification

Data type	Sensitivity	Handling
Uploaded inspiration images	High — may depict the user or third parties (celebrities/influencers)	Feature-extraction-only; not redistributed or shared with retailers; deleted at session end unless explicitly saved
Occasion / budget text	Medium — financially and personally revealing	Encrypted in transit and at rest; excluded from upstream model-training loops via enterprise API agreements
Size profile	Medium — body-adjacent but user-declared, never inferred	Stored only as the user's explicit selection; no biometric or body-shape inference is performed on images
Ratings, clicks, session events	Low–Medium — behavioral, not identity-revealing on its own	Pseudonymized in the analytics warehouse; joined to identity only where operationally necessary

9.2 Technical Controls

- **Encryption:** TLS 1.3 in transit; AES-256 at rest for object storage and database volumes.

- **No biometric or body-shape inference:** images are analyzed strictly for garment identity, color, and silhouette; a face-region exclusion step runs before any image reaches garment-detection processing.
- **Retention & deletion:** uploaded images carry a short TTL and are purged automatically; a user-triggered delete-on-request job removes images and history within a defined SLA.
- **Secrets management:** API keys and credentials (retailer feeds, LLM providers) held in a managed secrets store (AWS Secrets Manager / HashiCorp Vault), never in application config or source control.
- **Access control:** role-based access for internal tooling (catalog ops, support); production data access is logged and audited.
- **Dependency & vulnerability scanning:** automated scanning of container images and third-party packages in CI.
- **Model-provider data agreements:** all connections to hosted vision/LLM APIs use enterprise-tier, no-training-on-input agreements, consistent with the AI Feature document's privacy-first exclusions.

9.3 Content & IP Safety

- A content/IP safety pass runs on every candidate outfit before display, as part of the Budget & Safety Filter (Section 5, Step 7) — MAISON recommends and retrieves similar purchasable items rather than reproducing or redistributing the uploaded inspiration image itself.
- Legal review of the image-upload and outfit-recreation flow is a hard gate before the first AI-touching (non-Wizard-of-Oz) user, per the PRD's launch checklist — this is a process control, not a purely technical one, but the architecture supports it by never persisting or re-serving the original uploaded image beyond the session.

9.4 Monetization-Trust Control

Commission rates are normalized across all partnered retailers as a condition of the Retailer Partner Program, and the ranking policy is published in plain language on the comparison view (“ranked by fit, style match, and price — not by which retailer pays us”). Technically, this means the scoring engine in Section 8 has no retailer-identity or commission-rate term in its weighted formula — commission cannot enter the ranking even accidentally.

9.5 Regulatory Context (India — DPDP Act)

India's Digital Personal Data Protection Act, 2023, with its implementing Rules notified in November 2025, is being brought into force in three phases: the Data Protection Board was established immediately upon notification; a Consent Manager framework becomes operative roughly a year later; and the full suite of substantive obligations — notice, consent, breach notification, and data-principal rights — becomes enforceable roughly eighteen months after notification. Practically, this means MAISON should treat the 2026 build period as the window to implement DPDP-aligned practices — standalone privacy notices, verifiable consent capture, a 72-hour breach-notification runbook, and data-principal access/deletion tooling — rather than waiting for the enforcement deadline to catch up. This is a compliance program alongside engineering, not a substitute for the legal review already called out in Section 9.3; a qualified legal review of MAISON's specific data flows is recommended before scaling past validation.

10. Scalability Strategy

10.1 Stateless Application Tier

- All core application services (Session, Upload, Orchestration, Catalog Sync, Feedback) are stateless and horizontally scalable behind the API Gateway — deployed on Kubernetes or a managed container platform (ECS/Cloud Run) with autoscaling on CPU/queue-depth.

10.2 Data Layer Scaling

- **PostgreSQL:** read replicas for outfit/session reads; connection pooling (PgBouncer) as concurrent sessions grow; partitioning of high-volume tables (events, ratings) by date if they migrate into Postgres before a dedicated warehouse is justified.
- **Qdrant:** HNSW index scales well into the tens of millions of vectors on a single node; move to Qdrant's native sharding/replication once the catalog crosses roughly 1–2M SKUs or query latency degrades.
- **Redis:** cluster mode once cache/queue throughput exceeds a single node; the cache is not a system of record, so scaling it is operationally low-risk.
- **Object storage & CDN:** product images served through a CDN (Cloudflare/CloudFront) rather than the origin store, regardless of scale.

10.3 AI Inference Scaling

- **Batched, queued inference:** self-hosted Qwen models served via vLLM or TGI (Text Generation Inference), which batch concurrent requests for GPU efficiency far beyond naive one-request-per-call serving.
- **Burst capacity:** serverless GPU providers (Modal, RunPod, Together AI) absorb traffic spikes without over-provisioning always-on GPUs — particularly useful around GTM launch spikes (Section 11 flags the cost trade-off).
- **Embedding cache:** hot product embeddings and frequent style-profile embeddings are cached in Redis to avoid recomputation on repeat queries.

10.4 Catalog & Retailer Feed Scaling

- New retailer feeds are added as independent sync workers writing into the same normalized schema — the PRD's non-functional requirement that adding a retailer must not require changes to core outfit-generation logic is satisfied because Retrieval only ever queries the normalized catalog, never a retailer-specific format.
- Queue-based decoupling (SQS/Kafka) between feed ingestion and catalog writes prevents a slow or flaky retailer feed from blocking the rest of the sync pipeline.

10.5 Resilience

- **Circuit breakers:** around the AI orchestration path and each retailer feed integration, so one degraded dependency cannot cascade into a full outage.
- **Graceful degradation:** pre-curated, occasion-based fallback outfit sets (NFR4) are always warm and ready to serve, independent of AI/catalog health — this is a scaling control as much as a reliability one, since it caps worst-case latency under load.
- **Multi-region:** deferred until international expansion is on the roadmap; the MVP architecture is single-region but designed so services can be replicated per-region without schema changes.

11. Cost Analysis (MVP vs. Scale)

Figures below are directional planning estimates in USD, not vendor quotes — intended to support build-vs-buy and self-host-vs-API decisions, not a procurement budget.

11.1 Assumptions

- **MVP scenario:** ~5,000 outfit-generation sessions/month (roughly the Wizard-of-Oz → early-beta scale implied by the 15–20-participant validation study).
- **Scale scenario:** ~500,000 outfit-generation sessions/month (established product-market fit, multiple acquisition channels live per the GTM plan).

11.2 Monthly Cost Estimate by Component

Component	MVP (~5K sessions/mo)	Scale (~500K sessions/mo)	Notes
LLM inference (NLU + reasoning + explanation)	$150–$400 (hosted API, pay-per-token)	$1,500–$4,000 (self-hosted Qwen on reserved GPUs, or batched hosted API)	Crossover to self-hosting typically pays back once monthly token volume passes a low-to-mid five-figure dollar equivalent in API spend
Vision / embeddings (CLIP + garment detection)	$50–$150 (hosted API or small shared GPU)	$400–$1,200 (dedicated GPU instances, autoscaled)	Fashion-CLIP self-hosting is cheap even at scale; garment detection is the pricier of the two
Vector DB (Qdrant)	$0–$50 (self-hosted on a small VM)	$300–$800 (managed cluster or larger self-hosted nodes)	Cost driven by catalog size (SKU count), not session volume
Relational DB (Postgres)	$0–$25 (managed free/starter tier)	$200–$600 (managed, with read replicas)	Supabase/ Neon free tiers comfortably cover MVP
Cache / queue (Redis)	$0–$10	$100–$300	Serverless pricing keeps MVP near-zero
Object storage + CDN	$5–$20	$150–$400	Short image TTL keeps storage volume low even at scale

Component	MVP (~5K sessions/mo)	Scale (~500K sessions/mo)	Notes
Analytics warehouse	$0–$20 (pay-per-query, low volume)	$200–$600	BigQuery/ ClickHouse pay-per-query stays cheap unless dashboards run very frequently
Compute (app services, gateway)	$50–$150 (small managed containers)	$800–$2,000 (autoscaled container fleet)	Stateless services scale close to linearly with traffic
Retailer / affiliate feed access	$0–$100 (affiliate network fees, if any)	Typically revenue-share, not a hard cost	Most affiliate programs are commission-based, not fixed-fee
Monitoring & observability	$0–$30 (free tiers)	$150–$400	Logs/metrics/ tracing volume scales with request volume
Approximate total	$250–$950 / month	$3,800–$10,300 / month	Wide ranges reflect hosted-API vs. self-hosted choices in Sections 4 and 10

11.3 The Build-vs-Buy Lever

The single largest cost lever is whether LLM/vision inference runs on hosted, pay-per-token APIs or on self-hosted open-weight models (Qwen, Fashion-CLIP). Hosted APIs minimize upfront engineering effort and are the right choice through Wizard-of-Oz and early MVP; self-hosting becomes materially cheaper per-session once volume is high and consistent enough to keep GPU utilization reasonable. This is exactly why Section 4 recommends starting hosted and migrating deliberately rather than choosing one path permanently on day one.

12. Technology Trade-offs

Decision	Option A	Option B	Choice for MAISON	Why
API style	REST	GraphQL	REST (v1)	Client data needs are shallow and well-defined at MVP; revisit if wardrobe/calendar features fragment query shapes
LLM hosting	Hosted frontier API (Claude/GPT)	Self-hosted open-weight (Qwen)	Hosted first, self-hosted at scale	Hosted wins on iteration speed pre-PMF; self-hosted wins on unit economics post-PMF (Section 11)
Final ranking logic	LLM-based ranking	Rule-based weighted scoring	Rule-based scoring	Deterministic, debuggable, cheap, and A/B-testable; LLM stays confined to understanding and narration (Section 4.5)
Vector DB	Qdrant (open source)	Managed alternative (Pinecone/Weaviate Cloud)	Qdrant, self-hosted first	Open source, strong payload-filtering support, no per-vector billing at MVP volume
Vision approach	Dedicated fine-tuned detector	Vision-language model (Qwen2.5-VL) or hosted API	VLM/hosted API at MVP, detector at scale	Faster to ship one model than two; precision investment deferred until misidentification is a measured problem
Catalog architecture	Live per-request retailer queries	Unified, pre-indexed catalog	Unified catalog	Querying multiple retailer sites per request is slow, inconsistent, and fragile; a synced catalog enables fast, consistent multi-retailer filtering
Data topology	Single general-purpose database	Polyglot persistence (Postgres + Qdrant + Redis + warehouse)	Polyglot persistence	Similarity search, transactional writes, caching, and analytics are different access patterns that don't share one engine well
Update mechanism (result delivery)	Simple polling	Server-Sent Events	SSE, with polling fallback	One-directional server → client updates fit SSE better than bidirectional WebSockets; polling remains a safe fallback for constrained clients
Fine-tuning approach	Full fine-tuning	LoRA / QLoRA (parameter-efficient)	LoRA/QLoRA	Materially cheaper compute, faster iteration, and preserves base-model general reasoning while specializing for MAISON's schema and vocabulary
Service architecture	Monolith	Microservices (as in Section 2/3)	Modular services from day one,	Clear ownership boundaries (Session, Upload,

Decision	Option A	Option B	Choice for MAISON	Why
			deployed together if needed	Orchestration, Catalog, Feedback) make later independent scaling possible without a rewrite, even if MVP deploys them on shared infrastructure

Document Notes

This TAD builds directly on the four MAISON strategy and product documents (Product Discovery & Strategy; PRD & Agile Backlog; AI Feature & Responsible AI; Metrics, Experimentation & GTM) and on the user-provided technical architecture notes. It should be revisited once real usage data (post-Wizard-of-Oz, post-MVP) is available to validate the cost, latency, and model-selection assumptions made here.